

Peach Cars case studies

Case Study: Assessing the CTO Candidate for Peach

Introduction:

Peach is a thriving online marketplace for locally used cars, with a technology stack comprising JavaScript (Vue.js), and Python (Django). As the CTO, you understand the pivotal role of technology leadership in the company's growth. The CTO candidate we seek will play a crucial role in shaping the technical strategy, providing coaching to the product team, and guiding the product's direction. This case study is designed to assess the strategic and technical prowess of the potential CTO for Peach.

Scenario:

You are the newly appointed CTO of Peach, and you have been introduced to the product manager and development team. Peach has seen tremendous growth in its user base and revenue. The product manager is keen to explore new features and enhancements to maintain this growth and keep the platform competitive in the online used car marketplace. As the CTO, you have to navigate both the technical and strategic aspects of the company to drive its success.

Instructions:

Please consider the following tasks and scenarios to showcase your ability to balance technical leadership and strategic thinking in the context of Peach. Provide brief answers to the tasks.

Section 1: Product Strategy & Coaching

Scenario:

Peach's product manager, Tom, is eager to introduce new features and improvements to the website. He approaches you for guidance on the technical feasibility, potential impact, and prioritization of these ideas. As the CTO, you need to coach Tom on how to align technical and product strategies.

Tasks:

1. Review the product manager's feature ideas.

The Product manager proposed the following ideas:

- Automatically detecting errors in the data captured by inspection process
- Optimise image sizes to improve data consumption for website visitors
- Schedule a viewing from the website
- Add model dropdown to search and filter on website
- Add engine type dropdown to search and filter on website
- Automatically merge duplicate inventory
- Auto valuation tool
- Automated logbook capture and storage
- Tracking logbook transfers and modifications

Case Study: Assessing the CTO Candidate for Peach 1

2. Provide feedback on the technical feasibility and potential impact of each feature.

To establish a simple way to compare across features, feasibility and potential impact are used as criteria with scales:

Feasibility: 1 - 5

Impact: S (1 pt), M (2 pt), L (4 pt), XL (10 pt)

Automatically detecting errors in the data captured by inspection process

Feasibility: 3

Autos are mostly standard SKUs with variants so knowing the type of vehicle enables a cheap way to verify the data captured.

Potential impact: L

Potential to free up resources otherwise required to detect and correct errors.

Optimise image sizes to improve data consumption for website visitors

Feasibility: 5

There is a lot of clarity on processing and serving various image sizes.

Potential impact: M

Website users would be able to access content at a lower rate of data.

Schedule a viewing from the website

Feasibility: 5

This is fully within the sphere of control of product and development teams

Potential impact: M

It adds another means for users to view inventory

Add model dropdown to search and filter on website

Feasibility: 5

This is fully within the sphere of control of product and development teams

Potential impact: L

It improves search and discovery of inventory on the website

Add engine type dropdown to search and filter on website

Feasibility: 5

This is fully within the sphere of control of product and development teams

Potential impact: L

It improves search and discovery of inventory on the website

Automatically merge duplicate inventory

Feasibility: 4

Since duplicates would be uploaded by the same user, detection should be within reach.

Potential impact: M

It improves content quality and removes the nuisance factor duplicates introduce.

Auto valuation tool

Feasibility: 3

There have been similar efforts done successfully and using a 3rd party reduces implementation risk.

Potential impact: M

It improves a step in the purchase process and makes it easier to assess affordability.

Automated logbook capture and storage

Feasibility: 4

Logbooks are largely standard documents and OCR technology is capable at extracting data

Potential impact: M

It reduces data entry errors and increases quality of data

Tracking logbook transfers and modifications

Feasibility: 2

There's not much information around this to estimate

Potential impact: L

It enables a clear chain of ownership and history

Fix long load time for make and model drop-downs to load on first load

Feasibility: 5

The fix is fully within the sphere of control of the product and development teams

Potential impact: L

It fixes issues which might prevent some users from completing the sell a car flow.

3. Prioritize the features, considering technical constraints and user value.

Feature prioritization

Aa Name	Feasibility	Impact	Priority	# Total points
<u>Automatically detecting errors in the data captured by inspection process</u>	3	L	Medium	7
<u>Optimise image sizes to improve data consumption for website visitors</u>	5	M	Medium	7
<u>Schedule a viewing from the website</u>	5	M	Medium	7
<u>Add model dropdown to search and filter on</u>	5	L	High	9

Aa Name	Feasibility	Impact	Priority	# Total points
<u>website</u>				
<u>Add engine type dropdown to search and filter on website</u>	5	L	High	9
<u>Automatically merge duplicate inventory</u>	4	M	Medium	6
<u>Auto valuation tool</u>	3	M	Low	5
<u>Automated logbook capture and storage</u>	4	M	Medium	6
<u>Tracking logbook transfers and modifications</u>	2	L	Medium	6
<u>Fix long load time for make and model dropdowns to load on first load</u>	5	L	High	9

4. Coach Tom on how to maintain a balanced approach between technical possibilities and user needs.

Hi Tom,

Thanks for the feature proposals. I've taken a read through the feature list you shared. Thanks for doing a great job putting that together.

Given that we have a number of great ideas but are not able to focus on all these ideas, I spent a bit of time rating the features in the list on technical feasibility and potential user value. I thought that could be a quick way for us to gain intuition on what to prioritise and what to leave in the backlog for now.

Using tools like this enables us to strike a balance between what is feasible with our team now and providing incremental value to our users as quickly as possible.

In terms of technical feasibility, you have direct access to the developer team and you can poll the team leads for rough estimates of technical feasibility or to generally talk through your ideas and how technically feasible they are. This should help

I think we can gain an appreciation of user value potential of a feature when we consider all that we know about the user: pain points, feature requests, bugs, must-haves, etc that we've gathered across analytics tools, surveys, A/B tests, etc. With that

context, we have heuristics which can help us to make some quick and dirty estimations of a feature's value to our user segments.

Let's spend some time next week working through a backlog review together so we can both get in some practice.

Section 2: Technical Strategy

Scenario:

Peach's backend infrastructure is evolving rapidly to accommodate the increasing user base and data volumes. As the CTO, you must outline a high-level technical strategy to ensure scalability, performance, and reliability.

Tasks:

1. Describe your vision for Peach's technical architecture, considering its growth.

Given the growth that Peach is experiencing, we can expect some outcomes in the near term:

- users might experience capacity issues with the product as usage overwhelms resources
- users would surface more bugs and issues as the product gets more extensively used
- users demand for additional features as they explore more of the product

To be able to respond to the side effects of growth, the architecture needs to evolve to be:

- available: presents a usable UX to the user
- scalable: able to service an increasing number of requests within SLA
- resilient: can withstand service failures in a graceful manner in time for resolution

Micro-services architecture presents a number of designs for achieving these architecture goals and has become popular and well understood in engineering. It provides a good foundation to base the technical architecture.

In particular, the backend as a service approach seems to make sense here. In this approach, there's often a shared data resource with multiple user facing applications.

Therefore, a backend for each user facing application is built to isolate one front end from others. This allows product teams to fully own the entire stack for their application (frontend + backend) while relying on a common set of micro services.

Since a product already exists that's serving users, it would be better to stagger any transition to a new architecture, especially when moving to micro-services. This prevents disruptions to the user experience while the "backend" is being incrementally transitioned. Here, techniques like Strangler pattern enables the features to migrate one feature at a time as the move from the existing architecture to the new architecture unfolds. This is a safe and comfortable way to gradually replace the core of a running application without disrupting users.

To achieve this technical vision, the human and process layers need to grow and change to support the realisation of the vision.

Clearly defined processes for performing tasks need to be jointly developed, communicated and adopted to ensure good coordination and hand-off. This provides a safe and known context for team members to work.

Engineering (development), Delivery (SRE) and Growth (product) functions should be clearly identified. Joint spaces where all 3 functions can collaboratively review user feedback, feature requests, bugs, backlog can help create conditions for a team that's pulling in one direction.

2. Highlight potential challenges and solutions in the current tech stack (JavaScript (Vue.js), Python (Django))

When used properly, Vue.js and Django can provide a great tech stack to build the product with. However, there can also be some challenges:

- both vue.js and Django have steep learning curves for developers new to the stacks
- vue.js apps can grow to become slow when UX becomes complex
- Django's ORM can perform slowly with growing data sets

Solutions:

- provide training resources for developers
- use all the tools available to offset any slowness or latency, including: caching, indexing databases columns, query optimizers, etc

- using component libraries like DaisyUI or RadixUI to reduce complexity when working with components

3. Discuss the advantages and disadvantages of adopting technologies and architecture.

Advantages of adopting technologies and architecture include:

- easier best practices adoption: architectures encode best practices in an easy to use way if used properly
- faster with better quality: using architectures enables developers to create more quality output in a shorter amount of time

Disadvantages of adopting technologies and architectures include:

- junk: architectures and technologies are often used en masse and this can lead to the addition of unwanted features or behaviours
- slow: this can also slow down dev teams if the architecture or technology introduces unnecessary technical overhead

4. Explain how you plan to ensure data security and compliance.

Data security and compliance are key to organisations handling user data. Regulators have emphasised the importance of treating data in line with regulatory and compliance requirements.

Data security can be looked at in two states:

- at rest: data in storage (database, files, etc)
- in flight: data in transit between devices

When data is at rest, the key security considerations are: software access control, physical security, remote user access and encryption. Factors to consider when data is in flight include using SSL and VPN tunnels to prevent man-in-the-middle attacks, and other threats.

To ensure data security and compliance:

- use SSL for all services
- load all pages over HTTPS

- apply a strict ACL (access control list) for all services in delivery stack to prevent unauthorised access
- encrypt data at rest (in a database, mainly)
- domicile data as required by country regulations
- change all default system passwords
- expose only required ports on servers
- run all services in delivery stack within a VPC (virtual private cloud)
- create and enforce physical security policy: this prevents unauthorised access to physical infrastructure
- train staff to be aware of social engineering and other attacks

Section 3: System Optimization

Scenario:

Peach's website experiences occasional performance issues, leading to user dissatisfaction. You are tasked with identifying and resolving these issues to ensure a seamless user experience.

Tasks:

1. Describe your approach to identifying and diagnosing performance issues.

Performance metrics form the basis of spotting performance issues. My approach involves setting up dashboards to track relevant performance metrics along with alerts that trigger when any thresholds are breached. This enables proactive service management and ensures that users experience minimal downtimes and service degradations.

Logs form the basis of diagnosing performance issues. Streaming logs from all services in the delivery stack enables the SRE team to easily assess the state of any service within the delivery stack and to identify any anomalies quickly. Further work can then be done on the isolated service for quick resolution.

2. Suggest potential optimisations to enhance the website's speed and responsiveness.

- cache frequently accessed data such as car make and model

- generate various image sizes for different parts of the site: for instance, the landing pages can load smaller car images while a detail page loads larger images
- compress images to reduce size
- serving users closer to where they are by using CDNs: in this case, since most Peach users are in Kenya, the website can be served from a PoP in Kenya to reduce latency
- ensure website files such as JS and CSS files are minified to reduce payload sizes when website is loaded: this can improve time to first paint, which is a critical SEO metric
- lazy loading fetches data from the server only when it's needed on screen: this can reduce the total amount of data that's fetched from the server and can enhance the perception of speed on the website

3. Provide a brief technical explanation of how caching mechanisms work and their benefits in this context.

Caching speeds up data access by moving data close to the point of use.

Reading data from disk is expensive since software programs are executed in memory while data is stored on disk. The longer physical distance between disk storage (where data resides) and memory (where software resides) adds latency to data access and slows down the entire operation.

Moving chunks of data into memory moves the data closer to the software and shortens the distance between software and data, making for faster operations.

Caching, for example, car makes and models enables near instant response times when a user loads the sell a car page and fills in the forms. Currently, the make and model drop downs take on the order of minutes to load data.

4. Outline a plan to regularly monitor and maintain the website's performance.

The components of the plan are:

- set up website performance monitoring dashboard
- setup alerts for important performance metrics
- review performance metrics and alerts daily
- collect frequently occurring alerts or performance threshold breaches

- document and schedule fixes for issues causing recurring performance threshold breaches or alerts

This approach enables continuous improvement to the website as issues are constantly identified and resolved.

Section 4: Data-Driven Decision Making

Case Study: Assessing the CTO Candidate for Peach 2

Scenario:

Peach relies on data to understand user behavior, trends, and marketplace dynamics. The product manager seeks your guidance on implementing a data-driven approach.

Tasks:

1. Propose a strategy for collecting and analyzing user data to inform product decisions.

User data can come from various sources:

- collected in the background when users interact with our products
- directly from users
- from app store and other reviews

Usage data can be collected by adding user behaviour tracking tools to the website. This will track what users do on the website and presents the data as heat maps, session recordings, rage quits, etc.

Help desk and user support communications are a great way to collect data on user pain points. This source of data enables product teams to proactively research, document and schedule improvements that improve user experience.

In-person user research interviews are also a great way to collect direct feedback from users. The product and development teams can observe the users as they navigate using the features and document findings which can be used to improve the product.

2. Discuss the tools and technologies you would use to gather and process this data.

Website analytics

for tracking page views, bounce rates, time on site, conversion rates, etc. and detailed reports and insights into user engagement, traffic sources, user demographics.

Tools: Google Analytics, Adobe Analytics, Mixpanel

Session recording and heat maps

for viewing real-time user sessions, understanding how users interact with the website, identifying pain points, and improving the overall user experience

Tools: FullStory, HotJar, CrazyEgg

Data analytics

for analysing and querying user data, tracking trends

Tools: Metabase, Tableau, Power BI,

3. Explain how data insights can influence product development and strategy

Collecting and processing user data provides a detailed and unbiased view of user perception of our products. Reviewing usage data insights enables us to understand details like:

- features users need but don't work well: these are features we need to prioritise fixing
- features users don't care about: potential features to be removed or downgraded in importance
- bugs: fixing these issues should improve user satisfaction and NPS score

Access to usage data insights enables the product and development teams to deliver what matters to users and ensure that users are given a positive product experience when they use our products.

Section 5: Continuous Improvement

Scenario:

Peach's internal dashboard and vehicle assessment tool require frequent updates and improvements. As CTO, you must outline your approach to managing these tools.

Tasks:

1. Describe your strategy for handling the continuous improvement of internal tools.

Regular, scheduled interactions with users can help the product team to gather features requests and as well as bugs and other inconveniences. These issues can be

jointly explored by the product team and internal users and documented as part of the backlog.

Providing users with a means to file bugs and make feature requests makes it possible for users to share feedback with the product and engineering team.

2. Explain how you would balance feature development, bug fixes, and technical debt reduction.

The way I typically approach this is to split each development cycle into delivering new features, fixing bugs and reducing technical debt.

The specific time splits would be based on the nature of work that's planned for that cycle.

3. Share your thoughts on version control, continuous integration, and deployment processes.

These practices are required to enable teams to deliver at a high level so I believe in allocating resources to ensure that the team is equipped and trained on how to effectively use these tools in day to day work.

Automated code reviews powered by generative AI can improve developer productivity and also serve as a coach for developers by highlighting issues with commits and suggesting fixes with explanations.